Clean Feature Anchoring: Mitigating the Accuracy–Robustness Trade-off by Self-Distilling a Naturally Trained Model

Anonymous authors
Paper under double-blind review

Abstract

Adversarial training is among the most reliable defenses against adversarial examples, but it degrades standard accuracy, and this trade-off is the central obstacle to its use. The accuracy is not lost in general: a network of the same architecture trained naturally on the same data still holds it. We ask whether self-distillation can put it back, leaving robustness where it already is, in the inner maximization. Such a teacher is the extreme case of what adversarial distillation says goes wrong—the field diagnoses the target as too sharp and builds machinery to soften it, and a naturally trained network's maximum softmax probability is 0.820 against an adversarially trained student's 0.016. Softening does help, raising AutoAttack accuracy from 20.84 to 24.48 across a temperature sweep, and it leaves standard accuracy flat at 58 while the teacher holds 77.66: it repairs robustness, not the axis we came for. Reading the same teacher's feature instead recovers both. We anchor the student's adversarial feature to the teacher's clean feature, so the teacher is never evaluated under attack and transfers neither its instability nor any robustness, and prove that this single term is equivalent within a constant factor to fidelity plus local stability, with no coefficient between them. Combined with a per-sample attack radius set by the input sensitivity of the loss at fixed total budget, the method has no loss weight, no temperature and no trained classifier. The anchor is not merely compatible with robustness: with no weight averaging, no AWP and no radius rule, it leads label cross-entropy by 5.79 points of AutoAttack and 6.61 of standard accuracy under an identical schedule. At a fixed weight-averaging and AWP stack it matches the best published AutoAttack accuracy on CIFAR-100 to within 0.27 points while keeping 5.29 more points of standard accuracy, and transfers unchanged to CIFAR-10 and Tiny-ImageNet.

1 Introduction

Deep neural networks are vulnerable to adversarial examples, imperceptible perturbations of the input that change the prediction (Goodfellow et al., 2014; Madry et al., 2017; Carlini & Wagner, 2017), which raises concerns wherever such models are deployed in safety-critical systems (Ma et al., 2021; Grigorescu et al., 2020). Among the defenses proposed in response, adversarial training (Madry et al., 2017) remains the most reliable (Athalye et al., 2018; Croce & Hein, 2020): the model is optimized on perturbations produced by an inner maximization, and the robustness it attains has survived a decade of stronger attacks. Its cost, however, is standard accuracy. A network trained adversarially classifies unperturbed inputs considerably worse than the same network trained naturally, and the gap grows with the perturbation budget. This accuracy–robustness trade-off is partly intrinsic to the objective (Tsipras et al., 2018; Zhang et al., 2019) and it is the principal obstacle to deploying adversarially trained models in practice.

Two lines of work reduce it. The first modifies the target inside adversarial training. Hard labels are a poor supervisory signal for perturbed inputs, since they assert that every point

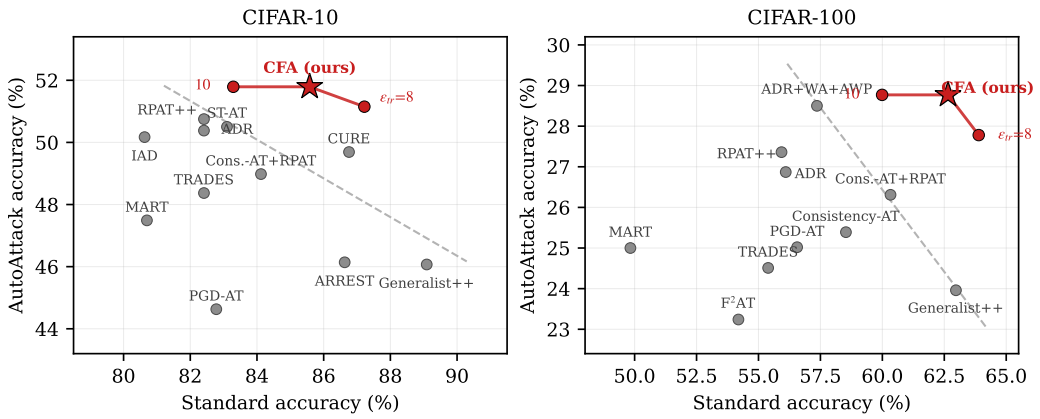

Figure 1: The accuracy–robustness frontier on ResNet-18, ℓ_∞, $\epsilon = 8/255$. Grey points are published results (Table 14); the dashed line is a least-squares fit through the Pareto-optimal ones and is drawn for reading rather than fitted for a claim. CFA is drawn as a curve rather than a point, because the training radius moves along the frontier at no cost; the star is the shipped configuration and the other markers differ from it only in ϵ_{train}. At $\epsilon_{\text{train}} = 8/255$ the curve strictly dominates Generalist++ on CIFAR-100 (63.89/27.78 against 62.97/23.96) and both CURE and ARREST on CIFAR-10 (87.22/51.15 against 86.76/49.69 and 86.63/46.14), so it is ahead on both axes rather than trading one for the other. Its teacher is a naturally trained network of the same architecture, obtained at no adversarial cost.

of an ϵ-ball belongs to one class with full confidence, and softening them alleviates both the trade-off and robust overfitting (Rice et al., 2020). Label smoothing (Pang et al., 2020), consistency regularization (Tack et al., 2022), and annealing self-distillation rectification (Wu et al., 2024) all replace the one-hot target with a smoother one, the last using the model's own weight average as the source. These methods require no additional network. The second line is adversarial distillation, which supplies the target from an adversarially trained teacher: ARD (Goldblum et al., 2020), IAD (Zhu et al., 2021), RSLAD (Zi et al., 2021), AdaAD (Huang et al., 2023) and IGDM (Lee et al., 2025) differ in which teacher quantity is matched and where the teacher is evaluated, and together they define the strongest reported accuracy–robustness frontiers for small architectures.

The two lines leave a gap between them. Adversarial distillation presupposes that an already-robust network exists, and in practice that network is a WideResNet adversarially trained with large volumes of generated data (Wang et al., 2023b)—costlier to obtain than the student it teaches. The robustness such methods report therefore originates outside the student, and what is being measured is the quality of the transfer. The teacher that is genuinely free, a naturally trained copy of the student's own architecture, is not used as a source of guidance for the adversarial term, and the reason is well founded: a naturally trained network attains its accuracy through features an adversary can manipulate (Ilyas et al., 2019), so transferring what it computes is expected to transfer that fragility as well. Three lines of work come close and are discussed in Section 2—ARREST (Suzuki et al., 2023) initializes from a standardly pretrained network and adds a representation-distance term to the label loss; DP-FAT (Cho & Kim, 2026) distills a natural teacher directly, rescaling its logits per sample so that only the class-discriminative direction transfers; and B-MTARD (Zhao et al., 2024) pairs a clean teacher with a robust one so that each handles the examples it is good at. In all three the natural network supplies logits and is balanced against another term or another teacher, and the balance is what has to be tuned.

Our question is about the other axis. Adversarial training discards standard accuracy that a naturally trained copy of the same network still holds. We ask whether that accuracy can be put back by distillation, and we leave robustness where it already is: in the student's own inner maximization.

The field has a diagnosis of what the teacher should supply, and it is half right. Adversarial distillation reads the one-hot target as too sharp and softens it — soft labels, smooth teacher logits, annealed and entropy-matched temperatures (Chen & Lee, 2021; Zi et al., 2021; Wu et al., 2024; Zhao et al., 2024). A naturally trained network is the extreme point of that idea, with a maximum softmax probability of 0.820 against an adversarially trained student's 0.016. Softening does work, and it works on the wrong axis: across a temperature sweep it lifts AutoAttack accuracy from 20.84 to 24.48 while leaving standard accuracy flat near 58, with the teacher holding 77.66 the whole time. The accuracy we came for does not survive the teacher's own softmax. It survives one layer earlier, and that is where we put the anchor.

The student's feature at the adversarial input is matched to the teacher's feature at the clean input, and that is the entire objective for the backbone. The label term is removed rather than supplemented, which is the difference between this and the methods that add a representation term to a loss they keep. Two consequences follow from the form alone. The teacher is never evaluated inside the perturbation ball, so nothing it does there can enter the optimization — and, by the same token, it cannot be a source of robustness. All robustness is produced by the student's inner maximization, exactly as in standard adversarial training. None of this says a natural teacher is preferable to a robust one; a robust teacher is better guidance for robustness, obtaining one is the expensive part, and we do not compete with methods that have one.

Deleting the label term would be a poor trade if the single remaining term only held the student near the teacher. It does more than that. Writing F for the student's clean deviation from the teacher and O for the diameter of the student's own output over the ϵ-ball, the anchor loss L satisfies $L \leq F + O \leq 3L$: minimizing one quantity controls teacher fidelity and local stability together, within a constant factor, and with no weight to balance between them. The bound uses only the triangle inequality, so it holds for the networks we actually train rather than for a model of them.

A second finding concerns what the teacher supplies. Across a ladder of naturally trained teachers differing only in training length, teacher accuracy and student accuracy are almost perfectly anti-correlated ($r = -0.999$): the most accurate teacher produces the worst student. What the student inherits is the teacher's class geometry rather than its predictions, and the two are not monotone in one another. This makes the teacher's training length a control on the accuracy–robustness frontier that costs nothing to exercise and that a robust teacher does not offer, since a robust teacher's own training is the expensive part. The non-monotonicity itself is not peculiar to natural teachers: in the robust-teacher setting a more robust teacher is likewise reported to fail to improve, or to harm, the student, with the teacher's predictive entropy on a consistent subset of the data indicating which way it will go (Anonymous, 2026). Our ladder is the natural-teacher analogue, and what varies along it is class separation rather than accuracy.

Building on these observations we propose Clean Feature Anchoring (CFA). A naturally supervised copy of the student architecture is trained on the same data, the student is initialized from it, and the backbone is then trained with the clean-feature anchor alone. The classifier is inherited and never trained at all.

One component is added to that. The attack radius is optimized over pixels, but what the objective measures is a displacement in feature space, and a fixed pixel radius moves different examples by very different amounts. Sensitivity-matched ϵ redistributes the per-sample radius in inverse proportion to the input sensitivity of the loss, holding the total budget across the batch exactly fixed. Each example is then perturbed to a comparable displacement of the objective rather than a comparable displacement of the input, and because the total is unchanged, the comparison against a uniform radius is a comparison of allocation and not of attack strength.

What is left has no temperature, no loss weight and no auxiliary head. We apply an identical configuration to every dataset, changing only the dataset name and the teacher checkpoint.

On CIFAR-100 with ResNet-18, CFA reaches 62.17% standard accuracy at 28.86% AutoAttack accuracy (Figure 1). Against the strongest method at comparable robustness this is +5.29 points of standard accuracy, and against the strongest method at comparable stan-

dard accuracy it is +4.81 points of AutoAttack accuracy; no method in this setting exceeds ours on both axes. We further run the published distillation objectives on our own natural teacher, which is the teacher they would have in this setting: ARD, RSLAD and AdaAD reach 20.24%, 21.30% and 23.19% AutoAttack accuracy, all below plain adversarial training initialized at the same teacher (26.46%). The accuracy a natural teacher holds is therefore not recovered by treating it as a logit target. Our contributions are as follows:

- We show that a naturally trained network can guide adversarial training when it is asked only for standard accuracy. Evaluated at the clean input and used as the entire backbone objective, a feature anchor controls fidelity to the teacher and local stability of the student jointly, within a constant factor, as a single unweighted term—so the teacher's own fragility has no path into the optimization, and no coefficient trades the two properties off.
- We show that what a naturally trained teacher transfers is class geometry rather than accuracy, with teacher and student accuracy anti-correlated at $r = -0.999$ across teachers, which turns the teacher's training length into a trade-off control available only in this setting.
- We propose Clean Feature Anchoring together with sensitivity-matched ϵ, a method with no loss weight, no temperature and no per-dataset tuning. Robustness remains attributable to the student's own inner maximization rather than to the teacher, but what that maximization ascends is not incidental: with no weight averaging, no AWP and no radius rule, the anchor alone leads label cross-entropy by 5.79 AutoAttack under an identical schedule.
- We run the published distillation objectives on the same natural teacher and find all of them below plain adversarial training from that teacher, which locates the difficulty in the logit target rather than in the teacher.
- We demonstrate a new accuracy–robustness frontier on CIFAR-10, CIFAR-100 and Tiny-ImageNet, and show the gain is not weight averaging or AWP: with weight averaging alone and half the schedule, CFA already exceeds the best published result that uses both.

2 Related Work

To be written.

3 Analysis

Adversarial training trades clean accuracy for robustness, and this section asks where in its construction the trade sits. The answer we arrive at is that it is in the target: a label asserted over a whole ball is a demand that cannot be met without distorting the clean prediction, and replacing it with a fixed point in feature space turns two competing objectives into one. That claim is what Proposition 1 makes precise, and it is also what tells us the anchor cannot cost robustness — which is a weaker statement than the paper needs, and we say so where it matters.

3.1 Preliminaries

Let $f = h \circ \Phi$ be a classifier with feature map Φ and linear head h. Adversarial training (Madry et al., 2017) solves

$$\min_{\theta} \; \mathbb{E}_{(x,y)} \max_{x' \in B(x,\epsilon)} \ell\big(f_\theta(x'), y\big), \qquad B(x,\epsilon) = \{x' : \|x' - x\|_\infty \le \epsilon\}. \tag{1}$$

The supervisory signal is the label, and it is asserted uniformly over $B(x,\epsilon)$: every point of the ball must reach full confidence in one class. That is where the accuracy cost comes from. We write $f_t = h_t \circ \Phi_t$ for a network of the same architecture trained naturally on the same data—the teacher throughout, frozen—so that $\Phi_t(x)$ is its feature and $f_t(x)$ its logits, and ask what it can supply.

3.2 Where a Natural Teacher Fails, and Where It Does Not

A method that consumes a teacher can go wrong in two independent places: where it reads the teacher, and what it reads from it. Both defects are present when a naturally trained network is used the way adversarial distillation uses one, and they can be measured separately.

What is read: the field's prescription is softening, and it is only half a fix. Four separate lines of work converge on the same diagnosis—the target is too sharp—and each builds its own machinery to soften it. LTD (Chen & Lee, 2021) argues that one-hot labels are the wrong supervision for adversarial training and replaces them with soft ones. RSLAD (Zi et al., 2021) makes the smoothness of the teacher's logits its central claim. ADR (Wu et al., 2024) draws its target from a weight average and scales it by a softmax temperature that starts near-uniform and is annealed down over training. B-MTARD (Zhao et al., 2024) formalizes the quantity being controlled—writing the teacher's knowledge scale as $\log C - H(P_T)$, so that a sharper teacher carries a larger one—and adjusts each teacher's temperature online to equalize it. In each of these the operative object is $\mathrm{softmax}(z_t/\tau)$ and the design question is τ.

Not every method that consumes a natural network poses it, and Section 2 places the exceptions; LBGAT (Cui et al., 2021) in particular matches logits under a squared error with no softmax at all, at the cost of training its natural branch jointly.

By that account a naturally trained teacher is the extreme case, and the measurement agrees: on clean test data its maximum softmax probability is 0.820, against 0.016 for the adversarially trained student it is supposed to teach. It is asking for a confidence twenty times sharper than the student's own.

The prescription follows, and it works. Sweeping τ on that teacher's logits in a fixed base regime—50 epochs, no weight averaging, no AWP, $\epsilon = 8/255$—AutoAttack accuracy rises from 20.84 at $\tau = 1$ to 24.48 at $\tau = 4$, then turns over. Sharpness is a genuine defect and softening genuinely repairs it.

target	Clean	AA	NRR
teacher logits, $\tau = 1$	58.26	20.84	30.70
teacher logits, $\tau = 4$	59.39	24.48	34.67
teacher logits, $\tau = 16$	57.78	24.00	33.91
teacher feature	62.72	25.88	36.64

What it does not repair is the accuracy. Across the whole sweep clean accuracy moves between 57.78 and 59.39—it is flat in τ—while the teacher it is being distilled from has 77.66. Reading the same teacher's feature instead, with no temperature anywhere, gains a further 1.40 AutoAttack and 3.33 clean, at the same read point and under the same regime.

The two numbers separate cleanly. Temperature moves robustness and leaves accuracy where it was; the feature moves both. Sharpness is therefore a real problem with a real fix, and it is not the problem this paper is trying to solve: the accuracy a naturally trained network holds does not survive its own softmax, and no choice of τ puts it back. It survives one layer earlier.

Where it is read: off the clean point the teacher has nothing to say. Under $\epsilon = 8/255$ the teacher's own feature rotates by 63.8° and its norm inflates by $\times 2.45$, so its output inside the ball is not a softened version of its output at the centre. The consequence is sharpest for methods that read it there. IGDM (Lee et al., 2025) matches a finite difference $f_t(x{+}\delta) - f_t(x{-}\delta)$ as a stand-in for the teacher's input gradient, which is a gradient only while the teacher is locally linear; its own diagnostic reports a Taylor remainder proportion of 0.012 for adversarially trained teachers. Applying that diagnostic to our checkpoints gives 0.0111 for an adversarially trained ResNet-18—reproducing their value, so the measurement is calibrated—and **0.4763** for the natural teacher, forty times further from linear. The target it constructs is correspondingly degenerate: $\|f_t(x{+}\delta) - f_t(x{-}\delta)\| = 14.14$ against $\|f_t(x)\| = 18.36$, with the softmax of that difference placing 0.917 of its mass on one class.

Together, they cost more than the teacher is worth. Table 5 gives the published objectives with our natural teacher in place of the robust one they assume, each at its own published recipe. Every one lands below plain adversarial training initialized from that same teacher—that is, below not distilling at all. The accuracy the teacher holds is real, and the logit route does not recover it.

What follows keeps the read point at x, where the second defect cannot arise, and takes the features, which the first measurement shows to be the better read.

3.3 What a Clean-Feature Anchor Controls

Consider matching the teacher's feature at the clean input against the student's feature at the adversarial input, that is, minimizing

$$\mathcal{L}(x) \;=\; \max_{x' \in B(x,\epsilon)} \big\|\Phi_s(x') - \Phi_t(x)\big\|^2. \tag{2}$$

Read literally, Equation (2) asks the student to reproduce a fixed vector from every point of the ball. That is a fidelity constraint, and a fidelity constraint alone would have no reason to produce robustness. It turns out not to be one, and the reason is visible once the quantity is compared with the two properties one would otherwise have to impose separately.

Fix x, write $B = B(x, \epsilon)$, and define

$$L = \max_{x' \in B} \|\Phi_s(x') - \Phi_t(x)\|, \tag{3}$$

$$F = \|\Phi_s(x) - \Phi_t(x)\|, \tag{4}$$

$$O = \max_{x', x'' \in B} \|\Phi_s(x') - \Phi_s(x'')\|. \tag{5}$$

Here L is the square root of Equation (2), the quantity the method minimizes; F is fidelity, how far the student is from the teacher at the clean point; and O is oscillation, the diameter of the student's own feature map over the ball. F is what adversarial training gives up relative to natural training, and O is what its label term buys. They are ordinarily two objectives.

Proposition 1. For every x and every $\epsilon \geq 0$,

$$L \;\leq\; F + O \;\leq\; 3L. \tag{6}$$

Proof. For the right-hand inequality, $F \leq L$ because $x \in B$, so the maximum in Equation (3) is at least the value at x. For O, insert $\Phi_t(x)$ between the two student features: for any $x', x'' \in B$,

$$\|\Phi_s(x') - \Phi_s(x'')\| \;\leq\; \|\Phi_s(x') - \Phi_t(x)\| + \|\Phi_t(x) - \Phi_s(x'')\| \;\leq\; 2L,$$

and taking the maximum over x', x'' gives $O \leq 2L$. Adding the two bounds gives $F+O \leq 3L$. For the left-hand inequality, insert $\Phi_s(x)$ instead: for any $x' \in B$,

$$\|\Phi_s(x') - \Phi_t(x)\| \;\leq\; \|\Phi_s(x') - \Phi_s(x)\| + \|\Phi_s(x) - \Phi_t(x)\| \;\leq\; O + F,$$

and taking the maximum over x' gives $L \leq F + O$. □

Only the triangle inequality is used, so Proposition 1 holds for the networks we actually train: no smoothness, no linearity, no assumption on the architecture or on the attack that realizes the maximum. Two consequences follow immediately.

Corollary 1. If the anchor is driven to $L \leq \delta$ at x, then $F \leq \delta$ and $O \leq 2\delta$ at x.

That is the operative statement: driving one scalar down bounds both properties, at rates that differ by a factor of two and by nothing else. Conversely, by the left inequality, L cannot be small unless both are, so the objective admits no solution that satisfies one at the expense of the other. A weighted sum $F + \lambda O$ has no such property for any fixed λ: its level sets contain points where either term is arbitrarily large. This is why the anchor is stated as the whole objective rather than as a regularizer, and why there is no coefficient to select.

Where the constant 3 comes from, and how tight it is. The factor is $1+2$: one from $F \le L$ and two from $O \le 2L$, and the second is attained when the student's outputs at the two extreme points of the ball sit on opposite sides of $\Phi_t(x)$ at distance L each. Our trained students are far from that configuration, which is the content of the measurement below.

The teacher appears at x and nowhere else. Equation (2) evaluates Φ_t at the clean point only, in the loss and in the attack that realizes the maximum. Whatever the teacher does inside B is therefore absent from the objective, and the instability measured in Section 3.2—63.8° of rotation, $\times 2.45$ of norm—has no path into the optimization. The same fact settles the symmetric question: since the teacher is never asked to resist a perturbation, it cannot supply robustness either, and all of it is produced by the student's inner maximization.

This is testable by moving the read point and changing nothing else, and the test is decisive. Replacing $\Phi_t(x)$ with $\Phi_t(x')$ in both the loss and the attack drives AutoAttack accuracy from 26.19 to 0.00 while clean accuracy rises to 76.11, within 1.55 of the natural teacher (Table 7). The failure is not that a moving target is harder to track. It is that $\|\Phi_s(x') - \Phi_t(x')\|$ is minimized exactly by $\Phi_s = \Phi_t$, so the student that copies a natural teacher everywhere is optimal—and it has no robustness at all. The clean read point is what makes Equation (2) a non-degenerate objective, and the same asymmetry that keeps the teacher's fragility out is what keeps the trivial solution out.

Verification. Proposition 1 predicts that a student trained on L ends up with a small O, and the prediction is checkable on the trained network. Measured over $\epsilon = 8/255$ balls on CIFAR-100, $L = 7.58$, $F = 6.30$ and $O_{\text{lb}} = 2.16$, against the teacher's own $O = 6.68$: the student is $3.1\times$ more stable than the network it was distilled from, on the same balls. Two things follow. The teacher's instability was not inherited, which is what the construction predicts. And $F \gg O_{\text{lb}}$, so what remains in the loss at convergence is fidelity rather than instability—the anchor is not being satisfied by making the student flat.

What the proposition is for, and what it is not. Proposition 1 says the anchor does not cost robustness: local stability is bounded by the same scalar as fidelity, so the two cannot be traded against each other. It is not a lower bound on how much robustness the objective buys, and we do not read it as one; Appendix B gives two partial results in that direction, each with the reason it falls short.

The measurement is stronger than the proposition, and it is stronger for a different reason than the proposition would suggest. Asked to work with neither weight averaging nor AWP nor a sensitivity-matched radius—the anchor alone, against label cross-entropy under an identical schedule, initialization and training radius—the anchor leads by 6.61 clean and 5.79 AutoAttack (Table 7). The obvious reading is that Proposition 1's bound on O is doing the work, and Section 5.4 shows it is not: the two students oscillate by the same relative amount under a matched attack. What differs is class separation, by a factor of 5.5, and that is where we locate the effect. For scale, the full recipe leads cross-entropy with weight averaging and AWP by 2.40. So the anchor is a source of robustness rather than a neutral passenger on one, and at matched stack it is the larger source; the components in Table 6 are additive on top of it rather than the origin of the effect. We keep Proposition 1 stated as the weaker claim because that is what the inequality supports—the 5.79 is measured, not derived.

4 Method

4.1 Clean Feature Anchoring

Adversarial training asks a model to be confidently right everywhere in a ball around every training point. That demand is the source of its accuracy cost, and softening it is what the trade-off literature spends its effort on. We remove the demand instead. The label is replaced, as the backbone's target, by a single fixed vector: the feature a naturally trained network of the same architecture assigns to the clean image. The ball no longer has to reach full confidence anywhere; it only has to land on that vector.

The teacher supplying the vector is not robust, and this is deliberate rather than a concession. It is what supplies a target that is accurate, and the robustness is left to the maximization that finds x_{adv} — an arrangement that only makes sense once the label loss is gone, since with a label loss present the two would compete for the same parameters. What the rest of this section states is the objective, three properties that follow from its form rather than from measurement, and the one place a hyperparameter survives: how large a ball each example gets.

Let Φ_t be the frozen feature map of a network of the same architecture trained naturally on the same data, and let h_t be its classifier. The student's feature map is initialized at $\Phi_s \leftarrow \Phi_t$ and trained with

$$\mathcal{L} \;=\; \mathbb{E}_x \big\|\Phi_s(x_{\text{adv}}) - \Phi_t(x)\big\|^2, \qquad x_{\text{adv}} \;=\; \underset{x' \in B(x,\epsilon)}{\arg\max} \big\|\Phi_s(x') - \Phi_t(x)\big\|, \tag{7}$$

where the inner maximization is realized by K steps of projected gradient ascent on the same quantity the outer problem minimizes,

$$x^{(k+1)} \;=\; \Pi_{B(x,\epsilon)}\Big(x^{(k)} + a \cdot \operatorname{sign} \nabla_x \big\|\Phi_s(x^{(k)}) - \Phi_t(x)\big\|^2\Big), \qquad x^{(0)} = x + \eta, \tag{8}$$

with η a small random initialization and Π the projection onto the ball. The target $\Phi_t(x)$ is a constant of this ascent: it is computed once, at the clean input, and does not move as $x^{(k)}$ does. Inner and outer problems therefore agree, as they do in standard adversarial training and unlike methods whose attack is generated against a different objective from the one being trained.

Equation (7) is the entire objective for the backbone. No label term, no logit term, and no coefficient joining them. Three properties follow from that form, and each is a statement about the construction rather than an empirical observation.

(i) The teacher never enters the perturbation ball. In both Equation (7) and Equation (8) the teacher is evaluated at x alone. Formally, the objective is a functional of Φ_t only through the single vector $\Phi_t(x)$, so two teachers agreeing at x and differing arbitrarily on $B(x,\epsilon) \setminus \{x\}$ induce the same optimization problem. The teacher's 63.8° of rotation under attack is exactly such a difference, and it is therefore invisible to training. The converse holds by the same argument: a quantity the objective cannot see cannot be a source of robustness either.

The restriction is load-bearing rather than convenient. Evaluating Φ_t at x' instead makes $\Phi_s = \Phi_t$ a global minimizer of Equation (7), and that minimizer is a natural model; measured, the student collapses onto the teacher at 76.11 clean and 0.00 AutoAttack (Table 7). Restricting the teacher to x is therefore not only what keeps its fragility out of the objective, it is what keeps the objective from having a degenerate solution.

(ii) The anchor is exactly zero at the clean point at initialization. Since $\Phi_s \leftarrow \Phi_t$, we have $\|\Phi_s(x) - \Phi_t(x)\| = 0$ for every x, so the loss is nonzero only because of the perturbation: every gradient the backbone receives at the start of training is attributable to the attack, which is the quantity to be trained. A logit target has this property only at temperature 1, where it carries no dark knowledge. It also has a consequence used in Section 4.2: at the first step the per-sample sensitivities are identical, so the radius allocation begins uniform and differentiates only as the student moves away from the teacher.

(iii) The classifier is inherited, and this is what a feature target buys. At test time the student is $h_t \circ \Phi_s$: the head is never trained. This is available because the target is written on the feature. A loss written on logits constrains only the composition $W\Phi$, so it determines the pair (Φ, W) up to the gauge

$$(\Phi, W) \;\longmapsto\; (A\Phi,\; WA^{-1}), \qquad A \in \mathrm{GL}(d), \tag{9}$$

and the same logits are reachable from a continuum of representations, none of which the teacher's classifier is calibrated for. Fixing the target at the feature removes that freedom and pins Φ_s to Φ_t itself, so h_t remains the right classifier for it. Table 9 shows this is

not merely permissible but preferred: every alternative we tried, including the best head temperature, scores below leaving the classifier alone. It is also what removes the last two hyperparameters, since a head distillation term would reintroduce a temperature and a weight.

4.2 Sensitivity-Matched ϵ

The problem a uniform radius creates. Equation (7) is optimized over a pixel ball while what it measures is a feature displacement, and the two are not proportional. The same ϵ moves the objective by an amount that varies across the dataset, and by a lot: logged over training, the coefficient of variation of $\|\nabla_x \mathcal{L}\|$ across a minibatch is 0.64. A fixed radius therefore delivers a strongly non-uniform amount of pressure—some examples are attacked hard in the geometry the loss lives in, others barely at all—even though every example is allotted the same pixel budget. Nothing in Equation (7) asks for that.

Equalizing the movement. Inside an ℓ_∞ ball of radius e the first-order change of any differentiable loss is

$$\max_{\|\delta\|_\infty \le e} \langle \nabla_x \mathcal{L}, \delta \rangle \;=\; e\, \|\nabla_x \mathcal{L}\|_1, \tag{10}$$

since ℓ_1 is the dual norm of ℓ_∞. Writing $g_i = \|\nabla_x \mathcal{L}(x_i)\|_1$, the movement induced at example i by a radius ϵ_i is $\epsilon_i g_i$ to first order, so equalizing it across a minibatch of N examples means imposing

$$\epsilon_i\, g_i \;=\; c \quad \text{for all } i, \qquad \text{subject to} \qquad \sum_{i=1}^{N} \epsilon_i \;=\; N\epsilon. \tag{11}$$

The constraint is what makes the rule comparable to a uniform one: it holds the total attack budget fixed. Solving Equation (11) gives $c = N\epsilon \big/ \sum_j g_j^{-1}$ and hence

$$\epsilon_i \;=\; N\epsilon \cdot \frac{g_i^{-1}}{\sum_j g_j^{-1}}, \qquad \text{generalized to} \qquad \epsilon_i \;\propto\; g_i^{-p}, \tag{12}$$

with $p = 0$ recovering the uniform radius and $p = 1$ the full equalization above. The implementation reads the gradient in ℓ_2 rather than the ℓ_1 derived here, for the reason Appendix D gives, and the choice is not free: the two land 0.29 AutoAttack apart. What the rule buys over a uniform radius survives it in direction and on the clean axis, and shrinks on the robust one — +1.75 clean and +0.44 AutoAttack at ℓ_2, +1.74 and +0.15 at ℓ_1. We ship the norm we measured to be better and derive the other, and Appendix D reports both. Equivalently Equation (12) is the max–min allocation: moving budget from the example whose objective moves most to the one whose objective moves least always improves the worst case, and the optimum is where no such move remains.

Keeping the radii in range. Left alone, Equation (12) assigns unbounded radii where $g_i \to 0$, which happens at the start of training by property (ii). We therefore constrain $\epsilon_i/\epsilon \in [0.5, 1.5]$ and solve the budget constraint inside that box, which Appendix D states exactly and which reduces to restoring the mean when no clip is active. We use $p = 1$ throughout and never tune it; Table 11 reports the $p = 0$ endpoint under the same recipe.

Why this is not the existing per-sample radius. Assigning a different ϵ per example is not new: IAAT (Balaji et al., 2019), MMA (Ding et al., 2020) and CAT (Cheng et al., 2022) all do it. They assign it from the difficulty of the example or from its margin in input space. Equation (12) assigns it from the input-sensitivity of the training loss—the geometry the objective is actually written in—and the two are not the same signal. Both are genuinely dispersed on the shipped recipe, and ours is the least dispersed of them: across training the sensitivity's coefficient of variation has median 0.56, the difficulty score's 0.75 and the logit margin's 1.33. The box of Appendix D compresses these before they reach the attack—the per-sample multiplier ends up with spread 0.32, 0.39 and 0.42 respectively—but the ordering survives it, and ours is at the bottom of it either way. What separates the rules is therefore not how much they spread the budget but which geometry they spread it along, and what follows compares competing allocations rather than an allocation against its absence.

The distinction is testable without confounding it with the budget. Holding the exact multiset of weights Equation (12) produces—same values, same clip, same mean, therefore the same total attack budget—and only reassigning which example receives which, ordered by difficulty as IAAT and CAT would, costs 0.80 AutoAttack and falls below even the uniform baseline on AutoAttack, CW and NRR alike. Swapping the signal outright rather than permuting it does the same: allocating by difficulty magnitude, the IAAT and CAT direction, gives 28.12, and by MMA's logit margin 27.79, against 28.42 for allocating nothing (Table 11). Every signal the existing family allocates from is worse than not allocating at all, and only the loss's own input-sensitivity is better. The same budget in a different order moves along the frontier; only this order moves the frontier itself.

Algorithm 1 states the procedure in full. Training the natural teacher is the only place labels appear and is ordinary supervised training; the anchored phase uses none, since the teacher vector ϕ_i is the entire supervisory signal, read once per example at the clean input, with the classifier carried through untouched. The method introduces no loss weight, no temperature and no auxiliary head, and the only quantity we vary between datasets is the teacher checkpoint.

Caveats. Equation (10) asks for the ℓ_1 norm and our runs use ℓ_2; both were trained and agree within noise. And g_i is a single backward pass at the clean x_i, so it is the sensitivity at the attack's starting point rather than along its trajectory.

5 Experiments

Four things have to be established, and they are not of equal weight. The first is the frontier claim itself: that the method is ahead on standard accuracy without giving up robustness, against baselines we run rather than quote. The second is that the accuracy a natural teacher holds is not simply available to anyone who distils from one — if the published objectives recover it when handed our teacher, the anchor is not the contribution. The third is the deletion: that removing the label term is what buys the gain rather than the schedule, the initialization, or the components stacked around it. The fourth is the mechanism, and it is where we report a negative result: the explanation we expected is measurably false, and the one that survives is about where features sit rather than how far they move.

5.1 Experimental Settings

Setup. CIFAR-10, CIFAR-100 (Krizhevsky et al., 2009) and Tiny-ImageNet-200 (Le & Yang, 2015), with random crop and horizontal flip only. For each dataset the teacher is a network of the same architecture trained naturally for 200 epochs on the same data (Table 1); no adversarial training enters its construction. The student is initialized from it and trained for 100 epochs with AdamW at learning rate 0.021 under a one-cycle schedule, batch size 128, with a 10-step PGD attack at step size 2/255 and training radius 8.8/255 (Section 5.5 varies it). Weight averaging ($\kappa = 0.999$, from 20% of the run) and an AWP proxy (Wu et al., 2020) ($\gamma = 0.005$, after epoch 10) are used and ablated in Section 5.4. Every dataset uses this identical configuration; only the teacher checkpoint changes. ResNet-18 throughout except Table 4.

Baselines and evaluation. Every baseline in Table 2 is run here rather than quoted: PGD-AT, TRADES and MART as standard adversarial training; ARD, RSLAD, AdaAD and IGDM as adversarial distillation, given our natural teacher in place of the robust one they assume; and RPAT++ reproduced from its own repository. Each row states in its Teacher column what it assumes available, cells we could not run are left empty with the reason, and published figures for all of them are collected in Table 14. We report clean accuracy and AutoAttack (standard version) on the full test set at $\epsilon = 8/255$, with Avg their mean and NRR their harmonic mean; per-attack values are in Table 13. All of our numbers are the final-epoch weight-averaged model, with no best-checkpoint selection.

Table 1: Teacher models. AutoAttack accuracy is 0 by construction: these networks never see an adversarial example.

Dataset	Teacher	Epochs	Clean	AA
CIFAR-10	ResNet-18, natural	200	95.32	0.00
CIFAR-100	ResNet-18, natural	200	77.66	0.00
Tiny-ImageNet	ResNet-18, natural	200	66.29	0.00

5.2 Main Results

Baselines are grouped by what they assume is available: (a) standard adversarial training; (b) adversarial distillation from an adversarially trained teacher; (c) methods guided by a naturally trained network, the family ours belongs to; and (d) current accuracy–robustness trade-off methods. Published numbers carry their source; '–' means the paper does not report that cell for this dataset and architecture.

WideResNet, CIFAR-10. At 88.67/55.29 the shipped recipe is strictly better on both axes than 30 of the 31 published WideResNet-34-10 results we collected (Table 14), and it takes both criteria: Avg 71.98 against the previous best 71.22 and NRR 68.11 against 67.53, both held by AT + WA + AWP, while its AutoAttack accuracy of 55.29 exceeds the highest published figure of 55.26. The single row it does not dominate is ARREST, which has 1.57 more standard accuracy and 5.09 less AutoAttack—a different point on the frontier rather than a better one. The comparison that matters most is with the two methods that also take a naturally trained network: against LBGAT we are +0.45 clean and +2.43 AutoAttack, and against ARREST −1.57 clean and +5.09 AutoAttack. Both of them pay for standard accuracy with robustness, and the anchor does not.

Reproductions, and one failure. RPAT++ we reproduce to within 0.31 AutoAttack of its published numbers on both datasets and report our own measurement. CURE we could not: seven runs of the official repository fall short on different axes, and Appendix H gives the detail. Its row is therefore left empty rather than filled from the paper. Published figures for every method here, including the ones we did not run, are collected in Table 14.

5.3 Published Distillation Objectives, Given a Natural Teacher

Every one of them lands below the row that does not distil at all: on both datasets, plain adversarial training initialized from the same teacher beats all four. The accuracy a natural teacher holds is not recovered by treating it as a logit target.

The IGDM row is reported at $\alpha = 1$ because its published $\alpha = 20$ does not train at all here: it collapses to chance on both datasets (1.00 on CIFAR-100, 10.00 on CIFAR-10), so ten classes did not rescue it. This is not a porting artifact. With a natural teacher the IGDM target $\mathrm{softmax}(f_t(x{+}\delta) - f_t(x{-}\delta))$ places 0.917 of its mass on a single class and the term reaches 17.3× the AdaAD loss. At $\alpha = 1$, which puts the two terms on comparable scales and is the most favourable reading available, it trains normally—and still finishes below AdaAD alone on both datasets, by 11.54 clean and 3.71 AA on CIFAR-100 and by 0.38 and 0.72 on CIFAR-10. Table 8 gives the structural reason: IGDM's finite difference is a gradient only while the teacher is locally linear, and a naturally trained teacher is forty times further from linear than the ones it was designed for.

5.4 Component Ablation

The sensitivity rule contributes +0.49 NRR at 50 epochs and +0.69 at 100 (+1.61/+0.21 and +1.80/+0.38 on the two axes) at identical total attack budget. AWP appears to change sign with schedule length — −0.25 NRR at 50 epochs against +1.08 at 100 — but the two blocks are not yet the same design, so that reading is held until the 50-epoch rebuild lands. And 50 epochs with weight averaging alone already exceeds ADR's full stack (38.46 against 38.08), at half the schedule and one fewer component, so the gain is not the stack.

Leaving the teacher's classifier alone beats every alternative, including the best temperature. At the full recipe the same holds—freezing the head scores 62.65/28.77/NRR 39.43 against 62.35/28.68/39.29 with the head-KD term retained—and this is what removes τ and β from the method. Both arms of that comparison predate the AWP correction described in Appendix I and are quoted as measured, so the pair is internally consistent; the corrected number for the frozen-head arm is the 62.17/28.86/39.42 reported everywhere else.

Where the anchor's robustness comes from. Table 7(a) moves Φ_t from x to x_{adv} in both the loss and the attack, and changes nothing else. Robustness is not degraded but eliminated: 0.00 under PGD-20, CW and AutoAttack alike, while clean accuracy rises 14.78 points to within 1.55 of the natural teacher. The reason is visible in the objective's value, which falls to 6.26 against roughly 50.7 for the anchor. Once Φ_t is evaluated inside the ball, $\Phi_s = \Phi_t$ minimizes Equation (7) exactly—and that solution is a natural model. Reading the teacher at the clean point is not a modelling preference; it is the only thing standing between this objective and a degenerate one.

What the anchor is worth on its own. The comparison against cross-entropy in Table 2 is not matched in stack: our row carries weight averaging and AWP and the cross-entropy row carries them too, but the anchor has never been asked what it does with neither. Table 7(b) asks. At the anchor's own base regime—same teacher initialization, same schedule, same training radius, no sensitivity-matched ϵ, no weight averaging, no AWP—replacing the anchor with label cross-entropy costs 6.61 clean and 5.79 AutoAttack. The anchor is therefore not merely compatible with robustness, as Proposition 1 alone would license; it is a source of it, and at matched stack the larger source. For scale, the shipped recipe leads cross-entropy with weight averaging and AWP by 2.40 AutoAttack.

Feature or logit: does the stack close the gap? Table 9 showed the feature anchor beating the best logit temperature in the base regime, by 3.33 clean and 1.40 AutoAttack. The obvious objection is that weight averaging is exactly what KD + SWA pairs distillation with, so the stack might be what our result rests on and the gap might close under it. If it did, the contribution would be the schedule and not the target, and this is the measurement that would say so. It does not close. Swapping only the loss in the shipped recipe — logit KL at the temperature that maximizes AutoAttack, everything else untouched — gives 56.97/26.96 against 62.17/28.86. The gap widens with the stack rather than closing, from 3.33/1.40 to 5.20/1.90, so the components amplify what the target does rather than substituting for it. The temperature also stops being a free parameter there: $\tau = 16$, which trails $\tau = 4$ by 0.48 AutoAttack without the stack, collapses to 42.45/20.10 with it. PGD-20 is the one metric that prefers the logit target (33.68 against 32.37), for the reason Appendix F gives: it ascends the cross-entropy through a head the logit objective has trained and the feature objective has left alone.

Where that robustness comes from, and where it does not. The natural explanation is local stability: Proposition 1 bounds the student's oscillation O by the anchor's own loss, so an anchored student should move less inside the ball than one trained on labels. We tested that and it is false. Attacking both models of Table 7(b) with the same true-label PGD and measuring the feature displacement they actually incur, the anchored student moves 0.275 of its own feature norm and the cross-entropy student 0.254, at cosines 0.9626 and 0.9671—indistinguishable, and if anything favouring cross-entropy. Proposition 1 is true and it is not the mechanism.

What separates them is where the features sit, not how far they travel. On the same models and the same attack:

	clean			adversarial		
	S_w/S_b	to own μ	to nearest μ	S_w/S_b	to own μ	to nearest μ
Anchor	0.898	28.1°	28.6°	2.681	33.3°	16.2°
Label CE	4.960	45.4°	21.6°	7.143	47.0°	18.3°

The anchored student's classes are 5.5× better separated on clean data and 2.7× better under attack, and it wins on both halves of the ratio: its clusters are tighter (28.1° against 45.4° to the class mean) and its class means are further apart (28.6° against 21.6°). The cross-entropy student is in the worse position by a margin that is easy to state: its samples sit twice as close to the nearest wrong class mean as to their own, 21.6° against 45.4°. Under attack the anchor's geometry degrades—0.898 to 2.681—and still ends up better than cross-entropy's undisturbed state.

This is the same quantity Section 5.5 finds varying along the teacher ladder, measured within a single pair of runs instead of across teachers, and it is what the anchor is for. A perturbation has to carry a feature across a boundary; separation decides how far that is, and the anchor inherits the teacher's separation because the teacher was never asked to be flat and so never spent its capacity on flatness.

One objection deserves its own measurement. Cross-entropy without weight averaging robustly overfits at this schedule, so part of the 5.79 could be decay rather than deficit. It does overfit—its PGD-20 peaks at 23.31 near epoch 70 and decays to 21.13—but the anchor does not: its PGD-20 rises monotonically to the final evaluation, $25.23 \to 28.86 \to 30.72 \to 31.47$. Measured against cross-entropy's best epoch rather than its last, the anchor's final still leads by 8.16. Not robustly overfitting is itself a property of the objective here, and it is the property weight averaging is normally bought to supply.

5.5 Training Radius, Teacher Length, and Larger Architectures

The training radius is a plateau, not a tuned point. Sweeping $\epsilon_{\text{train}} \in \{8, 8.8, 10\}/255$ with everything else fixed and evaluating at 8/255 throughout (Table 10), NRR peaks at 8.8/255 on both datasets but the curve is mild, and the standard 8/255 already exceeds the best published result on either (38.72 against ADR's 38.08; 64.48 against CURE's 63.19). Past 8.8/255 the radius buys nothing: AutoAttack and CW at 10/255 are identical to the last decimal to those at 8.8/255 on both datasets, while standard accuracy falls 2.66 and 2.29—the ϵ-independence Appendix B predicts above the threshold.

What the teacher transfers is geometry, not accuracy. Changing only the teacher checkpoint and leaving the student configuration untouched (Table 12), teacher accuracy and student accuracy are anti-correlated at $r = -0.999$ across five teachers trained for 50 to 300 epochs: the teacher gains 2.51 points and the student loses 2.04. NRR peaks in the middle, so the teacher's training length is a control on the operating point, and locating it costs five natural-training runs and no adversarial ones. The effect reproduces on Tiny-ImageNet where teacher accuracy is controlled to 0.32 points, so accuracy cannot be the cause; what varies monotonically is the teacher's class separation.

Larger architectures. WideResNet-34-10 at the same recipe on both CIFAR datasets is reported in Table 4, alongside the published WideResNet rows.

6 Conclusion

Adversarial training discards standard accuracy that a naturally trained copy of the same network still holds. We asked whether self-distillation can put it back without touching where the robustness comes from, and found that it can, provided the teacher is asked only for accuracy and is read only at the clean point. Anchoring the student's adversarial feature to the teacher's clean feature makes that precise: the teacher never enters the perturbation ball, so its own instability—63.8° of rotation under $\epsilon = 8/255$—has no path into the objective, and the single term bounds fidelity and local stability together to within a factor of three. The result is a method with no loss weight, no temperature and no trained classifier, whose robustness is produced by the student's own inner maximization rather than borrowed from the teacher, and which transfers across three datasets with only the teacher checkpoint changed.

Two findings sit alongside it. Consuming the same natural teacher the way adversarial distillation does—as a logit target, or read inside the ball—is worse than not distilling at all, on both datasets and for all four published objectives. And what the teacher transfers is its class geometry rather than its accuracy, with the two anti-correlated at $r = -0.999$ across training lengths, which turns the teacher's own schedule into a trade-off control that costs nothing to exercise.

Robustness is still produced by the student's inner maximization and by nothing the teacher supplies, since the teacher is never evaluated under attack. What we did not expect is that which objective that maximization ascends matters as much as it does. Stripped of weight averaging, of AWP and of the sensitivity-matched radius, the anchor alone leads label cross-entropy under an identical schedule and initialization by 5.79 points of AutoAttack as well as 6.61 of standard accuracy, and 1.80 of that robustness margin is traceable to attacking in feature space rather than through the label. For scale, at the full stack the anchor leads cross-entropy with weight averaging and AWP by 2.40. The anchor is therefore not a neutral passenger on machinery that supplies robustness; at matched stack it is the larger contributor, and the stack is additive on top. Proposition 1 licenses only the weaker half of this—that the anchor cannot cost robustness, since bounding one scalar bounds fidelity and local stability together instead of trading them. What the proposition does not give is the size of the effect, and a bound on that is the obvious next step; the measurements that would constrain it are reported here rather than set aside.

A Algorithm

Algorithm 1 Clean Feature Anchoring (CFA)

Input: dataset $\mathcal{D}$, radius ϵ, attack steps K, step size a, exponent p, box $[w_{\mathrm{lo}}, w_{\mathrm{hi}}]$
Output: robust student $h_t \circ \Phi_s$

{Phase 1 — natural self-teacher. The only stage that uses labels.}
Train $f_t = h_t \circ \Phi_t$ on $\mathcal{D}$ with standard cross-entropy; freeze it

{Phase 2 — anchored adversarial training. No labels, no head update.}
$\Phi_s \leftarrow \Phi_t$ {student initialized at the teacher}
for epoch $= 1$ to T do
  for minibatch $\{x_i\}_{i=1}^{N} \subset \mathcal{D}$ do
    $\phi_i \leftarrow \Phi_t(x_i)$ {teacher read once, at the clean input}
    $g_i \leftarrow \|\nabla_x \|\Phi_s(x_i) - \phi_i\|^2\|$ {one backward pass}
    $r_i \leftarrow (\bar{g}/g_i)^p$; solve $\sum_i \mathrm{clip}(tr_i, w_{\mathrm{lo}}, w_{\mathrm{hi}}) = N$ for t {Equation (18)}
    $\epsilon_i \leftarrow \epsilon \cdot \mathrm{clip}(tr_i, w_{\mathrm{lo}}, w_{\mathrm{hi}})$ $\{\sum_i \epsilon_i = N\epsilon\}$
    $x_i^0 \leftarrow x_i + \eta_i$
    for $k = 0$ to $K - 1$ do
      $x_i^{k+1} \leftarrow \Pi_{B(x_i, \epsilon_i)}\big(x_i^k + a \operatorname{sign} \nabla_x \|\Phi_s(x_i^k) - \phi_i\|^2\big)$
    end for
    $\mathcal{L} \leftarrow \frac{1}{N} \sum_i \|\Phi_s(x_i^K) - \phi_i\|^2$
    $\Phi_s \leftarrow \Phi_s - \mu \nabla_{\Phi_s} \mathcal{L}$ {h_t is never updated}
  end for
end for

B Two Partial Results on the Robustness Side

Guiding adversarial training with a naturally trained network is not new and we do not claim it: LBGAT, ARREST and DP-FAT all do it. What is different here is that they keep the adversarial label loss and add the natural teacher on top of it, so their robustness is bought by the label term and the anchor is free to contribute clean accuracy alone. Nobody needs to ask whether their teacher's fragility leaks into the student, because the term that makes the student robust is not the teacher.

We delete the label loss. Equation (7) is the whole backbone objective, so the robustness has to come from the inner maximization alone, against a target supplied by a network with 0 AutoAttack accuracy. That makes an otherwise idle objection load-bearing: a representation assembled out of features an adversary can flip is being handed to a student as the only thing it is trained on, and the student might reasonably be expected to inherit the fragility along with the answer. The first result below says that in the model where the objection is usually posed, it does not. The second is a certificate that tracks robustness without predicting it. Neither closes the question the paper actually sells — how much robustness the objective buys — and both are reported with the reason they fall short.

B.1 In the Two-Feature Model, the Non-Robust Coefficient Is Zeroed Rather Than Shrunk

The setting is the two-feature model of Tsipras et al. (2018), which is where the objection is usually posed. One feature x_1 agrees with the label with probability $p = 0.95$ and cannot be attacked. A further d features $z_j \sim \mathcal{N}(\eta y, 1)$ are individually almost uninformative, are jointly close to perfectly informative through their mean $m(z)$, and can all be moved by an ℓ_∞ adversary. A naturally trained network therefore reads $m(z)$: near-perfect clean accuracy, no robustness whatever. That is our teacher's profile, so $\Phi_t = m(z)$.

Write the student's feature as $\Phi_s = ax_1 + b\,m(z)$. Then b is exactly the quantity the objection is about: how much of the teacher's fragile route the student takes over. If the objection is right, minimizing our objective should leave b large.

Proposition 2. With the inner maximum exact, $J(a, b) = \mathbb{E}[R^2] + 2\epsilon|b|\,\mathbb{E}|R| + \epsilon^2 b^2$ with $R = ax_1 + (b-1)m$, kinked at $b = 0$ with subgradient jump $2\epsilon\,\mathbb{E}|R(a, 0)| > 0$; above a threshold ϵ_0 the minimizer has $b^\star = 0$ exactly.

Proof. The adversary reaches Φ_s only through m, and only through the mean perturbation $\bar{\delta} = \frac{1}{d}\sum_j \delta_j$, which ranges over $[-\epsilon, \epsilon]$ as $\|\delta\|_\infty \le \epsilon$. The inner objective is therefore $\max_{|\bar{\delta}|\le\epsilon}(R + b\bar{\delta})^2 = (|R| + \epsilon|b|)^2$, since a scalar quadratic is maximized at an endpoint and the sign of $\bar{\delta}$ is free. Taking expectations and expanding the square gives the display.

Fix a and differentiate in b. The term $\mathbb{E}[R^2]$ and $\epsilon^2 b^2$ are smooth, while $2\epsilon|b|\,\mathbb{E}|R|$ contributes $\pm 2\epsilon\,\mathbb{E}|R|$ as $b \to 0^\pm$. The one-sided derivatives at $b = 0$ are thus $2\mathbb{E}[R(a, 0)\, m] \pm 2\epsilon\,\mathbb{E}|R(a, 0)|$, so $b = 0$ is a minimum along b exactly when

$$\left|\mathbb{E}[R(a, 0)\, m]\right| \;\le\; \epsilon\,\mathbb{E}|R(a, 0)|. \tag{13}$$

At $b = 0$ the objective in a is $\mathbb{E}(ax_1 - m)^2$, minimized at $a^\star = \mathbb{E}[x_1 m]/\mathbb{E}[x_1^2] = q\eta$, and there $\mathbb{E}[R(a^\star, 0)\, m] = q^2\eta^2 - (\eta^2 + \frac{1}{d}) = -c$ with

$$c \;=\; \eta^2 v + \tfrac{1}{d}, \qquad v = 1 - q^2. \tag{14}$$

Equation (13) therefore holds for every

$$\epsilon \;\ge\; \epsilon_0 \;=\; \frac{c}{\mathbb{E}|q\eta x_1 - m|}, \tag{15}$$

which is the stated threshold. The mechanism is visible in Equation (13): the left side is fixed while the right grows linearly in ϵ, so the kink's slope overtakes the quadratic gain at a finite radius rather than asymptotically. □

Why the coefficient is deleted rather than shrunk. The adversary contributes $2\epsilon|b|\,\mathbb{E}|R|$ to the loss — linear in $|b|$, not quadratic in b. A quadratic penalty pushes a coefficient toward zero and never arrives; a penalty on $|b|$ is kinked at the origin and sets the coefficient to zero outright once its slope dominates, which is the same distinction that makes the lasso sparse and the ridge not. What produces the kink here is that the student is asked to reproduce a value rather than a route: the fragile features carry that value on clean data but not inside the ball, so past a certain radius carrying any of them costs more than it pays. The teacher supplies what to match, and the inner maximization discards how the teacher got there.

At $d = 512$ the model's constants give $c = 7.89 \times 10^{-3}$ and $\mathbb{E}|q\eta x_1 - m| = 0.0529$, hence $\epsilon_0 = 0.843\,\eta$ — inside the bracket $(0.5\eta, 1.0\eta)$ that the Monte-Carlo minimization returns, and far below the 2η at which the model is usually stated.

Two caveats are worth stating rather than leaving to be found. First, Equation (15) is a condition for $b = 0$ to minimize along b at $a = a^\star$; J is not jointly convex, because $|b|\,\mathbb{E}|R|$ is a product of two convex factors, so global minimality is what the numerical check below supplies and not what the argument proves. Second, the model assumes x_1 is unperturbable and z fully perturbable, so some suppression of z is built in before anything is shown. What is not built in is exactness, the threshold, and the ϵ-independence that follows it.

That the exactness is the kink's doing, and not a general consequence of adding an adversary, is visible from dropping it. With the $|b|$ term removed, $J_{\text{quad}} = \mathbb{E}[R^2] + \epsilon^2 b^2$; minimizing out a and writing $u = 1 - b$ collapses it to $u^2 c + \epsilon^2 (1-u)^2$, so

$$b^\star_{\text{quad}} \;=\; \frac{c}{c + \epsilon^2}, \tag{16}$$

which is strictly positive at every radius. Against the exact minimizer ($d = 512$, Monte-Carlo, $n = 2 \times 10^5$):

ϵ	$b^\star$ (exact)	$b^\star_{\text{quad}}$ (kink dropped)
0.5η	0.510	0.502
1.0η	0.000	0.202
2.0η	0.000	0.060
4.0η	0.000	0.016

The two columns agree below ϵ_0 and separate above it: the quadratic surrogate shrinks the non-robust coefficient and never deletes it, which is the ordinary picture, while the kinked objective sets it to zero and keeps it there.

Since every ϵ in J multiplies b, at $b^\star = 0$ the objective is $\mathbb{E}(ax_1 - m)^2$ and is ϵ-free—past the threshold a larger radius costs the anchor nothing, which matches the saturation measured in Table 10, where AutoAttack and CW at 10/255 are identical to the last decimal to those at 8.8/255 on both datasets.

This model rules out the paper's effect by construction, so Proposition 2 cannot be the explanation of it: its reliability spectrum has two atoms and no mass near $\eta_j/\epsilon = 1$, where the effect lives. It is a statement about the free half only.

B.2 What Robust Accuracy Tracks Is the Loss Measured Against a Margin

Writing $\gamma_t(x)$ for the teacher's margin at x and D_y for the relevant decision-boundary constant, Cauchy–Schwarz gives that $\gamma_t(x) > D_y \cdot L(x)$ implies the student is robustly correct at x. The certificate is vacuous at the scales we train at, but the ratio DL/γ_t tracks both axes across the teacher ladder of Table 12. What it does not do is predict the size of the effect, and it saturates before AutoAttack does.

C Additional Results

The tables below are referred to from the main text and are placed here for space. They are the same measurements under the same protocol; nothing is reported here that is not stated in the main body.

D The Allocation Rule and Its Budget Solve

D.1 Where the rule comes from

The objective is optimized over a pixel ball, while what it measures is a displacement in feature space, and a fixed pixel radius moves that displacement by very different amounts

across samples. The allocation follows from one elementary fact: for a differentiable loss the first-order movement available inside an ℓ_∞ ball of radius e is

$$\max_{\|\delta\|_\infty \le e} \langle \nabla_x \mathcal{L},\, \delta \rangle \;=\; e\, \|\nabla_x \mathcal{L}\|_1, \tag{17}$$

the dual norm of ℓ_∞ being ℓ_1. Equalizing that movement across a batch subject to a fixed total budget $\sum_i \epsilon_i = N\epsilon$ gives $\epsilon_i g_i = \text{const}$, that is $\epsilon_i \propto 1/g_i$ with $g_i = \|\nabla_x \mathcal{L}(x_i)\|_1$. The same allocation is the max–min one: if two samples move by different amounts, shifting budget from the sample that moves more to the sample that moves less raises the minimum, so no unequal assignment is optimal. Taking $\epsilon_i \propto (\bar{g}/g_i)^p$ makes $p = 1$ that allocation, $p = 0$ the uniform-pixel one, and $p \in (0, 1)$ an interpolation.

Two approximations are worth declaring rather than leaving to be found. First, g_i is measured once, by a single backward pass at the clean x_i before the attack starts, so it is the sensitivity at the starting point and not along the PGD trajectory. Second, everything above is first order in ϵ. Neither is repaired anywhere in the method; the rule is a budget heuristic with a derivation, not an exactly solved subproblem.

We also do not claim the rule is specific to a directional objective. $\|\nabla_x \mathcal{L}\|_1$ is defined for any loss and the implementation reads whichever loss is configured; on the shipped raw-ℓ_2 anchor it equalizes a feature displacement rather than an angle, which is why the honest name is sensitivity-matched ϵ rather than an angular budget. What separates it from IAAT, MMA and CAT is not directionality but the signal: they allocate from a property of the sample, this allocates from the geometry the training loss is written in.

One discrepancy is ours to declare rather than the reader's to find. Equation (17) gives the ℓ_1 norm and the implementation reads ℓ_2, for no better reason than that the cell which fixed the recipe used it and every logged run since had to stay reproducible against that choice. The two are not the same allocation. Measured over training on the shipped configuration, the ℓ_1 signal's coefficient of variation has median 0.68 against the ℓ_2 signal's 0.56, and the per-sample multiplier the attack receives has spread 0.34 against 0.32 — so the norms really do distribute the budget differently.

They nonetheless land close together. Training the shipped recipe with feat-dir_angeps_gnorm set to 1 and nothing else changed gives 62.16/31.97/30.56/28.57 (NRR 39.15) against 62.17/32.37/30.93/28.86 (NRR 39.42): clean accuracy is identical to a hundredth and AutoAttack differs by 0.29, in favour of the norm we ship.

We do not call that a tie. It is twice the seed-to-seed spread of Appendix E, and it is the same order of magnitude as what the allocation buys in the first place: against the uniform radius, $p = 1$ gains +1.75 clean and +0.44 AutoAttack at ℓ_2 and +1.74 and +0.15 at ℓ_1. So the rule's benefit survives the norm in direction and almost entirely on the clean axis, while most of its AutoAttack half does not. The honest statement is that the norm is a real choice which we made before we could compare, that the comparison favours what we ship, and that a reader preferring the derived ℓ_1 would still get the clean gain and a third of the robust one.

D.2 Keeping the radii in the box

With $r_i = (\bar{g}/g_i)^p$ and box $[w_{\text{lo}}, w_{\text{hi}}] = [0.5, 1.5]$, the radii that satisfy both the box and the budget exactly are $\epsilon_i = \epsilon \cdot \text{clip}(t r_i, w_{\text{lo}}, w_{\text{hi}})$ with t the solution of

$$\sum_{i=1}^{N} \text{clip}\big(t\, r_i,\; w_{\text{lo}},\; w_{\text{hi}}\big) \;=\; N. \tag{18}$$

The left side is continuous and nondecreasing in t from $N w_{\text{lo}}$ to $N w_{\text{hi}}$, so a root exists and bisection finds it; when no clip is active it reduces to $t = N/\sum_i r_i$, that is, to restoring the mean. Clipping first and rescaling afterwards, the obvious alternative, does not preserve the box—the rescaling moves the clipped entries back out of it.

E Experimental Details

Every number we report is the final epoch, never a checkpoint selected on a robustness metric. This matters when reading Table 14: RPAT selects on PGD-20 and ADR on PGD-10, so those rows are upper envelopes of a training run where ours is its endpoint.

Teacher. One naturally trained network per dataset, no adversarial examples anywhere in its training, and the student is warm-started from the same checkpoint. ResNet-18, SGD at learning rate 0.1 under a one-cycle schedule, batch size 128, no augmentation beyond the standard crop and flip: 200 epochs on CIFAR-10 and CIFAR-100, 80 on Tiny-ImageNet-200. The resulting teachers reach 95.33 (CIFAR-10), 77.66 (CIFAR-100) and the Tiny-ImageNet figure quoted in Table 3. The teacher's own robustness is 0 — it is never attacked and is not meant to survive attack; Table 12 is the ablation over how long it is trained.

Student. The three shipped cells differ only where the dataset forces them to.

	CIFAR-10	CIFAR-100	Tiny-ImageNet-200
Backbone	ResNet-18	ResNet-18	ResNet-18
Optimizer	AdamW	AdamW	AdamW
Learning rate / schedule	0.021, one-cycle	0.021, one-cycle	0.021, one-cycle
Epochs	100	100	100
Batch size	128	128	128
Training radius ϵ_{train}	8.8/255	8.8/255	8.8/255
Inner attack	PGD-10, step 2/255	PGD-10, step 2/255	PGD-10, step 2/255
Allocation exponent p	1	1	1
Radius box $[w_{\text{lo}}, w_{\text{hi}}]$	$[0.5, 1.5]$	$[0.5, 1.5]$	$[0.5, 1.5]$
Classifier head	frozen	frozen	frozen
Weight averaging (start, decay)	0.2, 0.999	0.2, 0.999	0.2, 0.999
AWP (γ, warmup)	0.005, 10 ep	0.005, 10 ep	0.005, 10 ep
Teacher checkpoint	clean, 200 ep	clean, 200 ep	clean, 80 ep

The head being frozen is what removes τ and β from the method: with the classifier inherited and never updated, the head-KD term of earlier drafts has no parameters to reach and is switched off entirely. p is not tuned. $\epsilon_{\text{train}} = 8.8/255$ is the one value swept, in Table 10, and evaluation is at 8/255 in every cell regardless.

Evaluation. AutoAttack, standard version (APGD-CE and APGD-T), ℓ_∞ at $\epsilon = 8/255$, run on the full test set. PGD-20 and CW_∞ are reported alongside because they disagree with each other in a way that matters (Appendix F), and NRR is the harmonic mean of clean accuracy and AutoAttack accuracy. Where a table reports a method we ported rather than quoted, the port runs on its own published recipe — HAT and LBGAT both collapse under our schedule and are given theirs — and only the evaluation is ours.

Seeds, and how large a difference has to be. Every number in this paper is one seed. The ablations are read as differences of a few tenths, so one repeat is reported to say what a tenth is worth: the shipped CIFAR-100 cell at seed 1 gives 62.00/32.49/30.82/28.74 against seed 0's 62.17/32.37/30.93/28.86, moving each metric by 0.11 to 0.17 and AutoAttack by 0.12. Differences at that scale are not read as differences anywhere in this paper; the component deltas of Table 6 and the gaps of Table 11 are between four and fifteen times it. One repeat is one repeat, and it bounds nothing — it is reported because a reader otherwise has no scale at all.

Compute. A single RTX 5080. One student cell is about 2.2 hours on CIFAR-100 at 100 epochs, of which the AutoAttack evaluation is roughly 7 minutes; Tiny-ImageNet is longer in proportion to its test set.

F Per-Attack Metrics

Table 13 gives every attack we measure for our own cells. The main tables report clean accuracy and AutoAttack because those are the two axes the trade-off is defined on, and because PGD-20 is not a faithful reading of this method.

Why PGD-20 is not the metric here. The student inherits the teacher's classifier and never retrains it. PGD-20 ascends the cross-entropy through that classifier, so it reads a head that was fitted for a different feature distribution; AutoAttack, which includes a targeted APGD, and CW_∞, which attacks the logit margin directly, do not depend on that fit in the same way. The consequence is measurable on a controlled pair. The first two rows of Table 13 share a backbone objective, a schedule and an attack, and differ only in whether the classifier is refit:

	PGD-20	CW	AA
head-KD term retained	36.26	30.65	28.68
head frozen	32.63	30.66	28.77
difference	−3.63	+0.01	+0.09

Changing the classifier moves PGD-20 by 3.63 points and moves CW and AutoAttack by hundredths. The same pattern recurs across our ablations—the head-temperature sweep, the label-smoothing sweep, the directional-versus-raw comparison, and the radius sweep all show PGD-20 moving against CW and AutoAttack. We therefore report PGD-20 for completeness and read the trade-off on clean accuracy and AutoAttack, which is also what ADR and ARREST report in their main tables.

G Published Results, for Context

Table 2 contain only numbers we measured ourselves. Table 14 collects the published figures for the same and neighbouring methods, so that our runs can be placed against the literature without the two being mixed in one column. These are quoted as reported and were produced by other codebases, other schedules and other checkpoint-selection rules—RPAT selects its best checkpoint by PGD-20 and ADR by PGD-10, where every number of ours is the final epoch—so they should be compared within a source rather than across.

Read against Table 2 and restricted to the ResNet-18 blocks, which are the ones our numbers are comparable with, our CFA row exceeds every published Avg and NRR on both datasets: 62.17/28.86 at Avg 45.52 on CIFAR-100 and 84.96/51.74 at Avg 68.35 on CIFAR-10.

The restriction is not a convenience. The WideResNet-34-10 block sits above us on both criteria— RPAT++ reaches Avg 70.87 and NRR 67.30 there—because a 48.3M network is a different model, not a better method, and Table 4 is where that comparison belongs. The same caution applies to the PreActResNet-18 blocks in the other direction: RPAT++ falls to 82.63/51.00 there, and reading that as a defeat for RPAT would be the identical error. We state all of this as a comparison across codebases and architectures rather than as a controlled result, which is what Table 2 is for.

H The CURE Reproduction

Seven runs of the official CURE repository, from its own code and our shared CIFAR data, none of which reproduce the published 86.76/49.69 on CIFAR-10. They fall short on different axes: the closest on standard accuracy reaches 86.11 at AA 40.65, the closest on robustness AA 45.63 at clean 81.15, while the published pair has both. Running the repository unmodified gives 65.03/29.74. Computing its RGP prominence mask on absolute values raises AA by 15.9 points (29.74 → 45.63), which localizes the discrepancy to that step without closing it.

We re-evaluated all five surviving checkpoints with our own pipeline and it agrees with the repository's own evaluation to within 0.05 AutoAttack on every one, so the gap is in training

rather than in measurement. For contrast, RPAT++ reproduces to within 0.31 AutoAttack of its published numbers on both datasets under the same conditions. The CURE row of Table 14 is therefore the paper's own number, and CURE is left empty in Table 2.

I A Correction to the AWP Proxy, and What It Moved

AWP ascends an auxiliary objective in weight space before each descent step. In our implementation the closure used for that ascent had drifted from the closure used for descent, in two ways. It retained a head-distillation term that freeze_head removes from training, so AWP was maximizing a quantity partly composed of a term the model never optimizes; and a detach intended for the normalized feature was applied on a path that feeds the unnormalized one. Measured on CIFAR-100 before the fix, the spurious term was 7.15 against the anchor's 4.15 and contributed 40.8% of the AWP backbone gradient, at cosine 0.918 to the correct direction. Separately, the head was frozen after the proxy was copied, which is harmless only because the proxy's first step is delayed by the AWP warmup.

We report this because it is the kind of defect that changes a paper's numbers, and because here it did not. Re-running the shipped recipe with both closures agreeing gives 62.17/28.86 against 62.65/28.77 on CIFAR-100 and 84.96/51.74 against 85.58/51.79 on CIFAR-10: NRR moves by 0.01 and 0.22, no ordering in any table changes, and clean accuracy moves more than robustness does. AWP is evidently insensitive to a large, well-aligned perturbation of its ascent direction, which is consistent with its role here as a regularizer of sharpness rather than a precisely targeted objective. All numbers reported for the shipped recipe are post-correction. Two exceptions are stated where they appear: the head-freeze comparison (in Section 5.4 and again in Appendix F), the component ladder in Table 6, and the ϵ_{train} sweep in Table 10 were measured before it. In each, every arm shares the defect, so the comparison is internally consistent even though its absolute values sit a few hundredths from the corrected ones.

References

Anonymous. Toward understanding adversarial distillation. In ICML, 2026.

Anish Athalye, Nicholas Carlini, and David Wagner. Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples. In International conference on machine learning, pp. 274–283. PMLR, 2018.

Yogesh Balaji, Tom Goldstein, and Judy Hoffman. Instance adaptive adversarial training. In arXiv, 2019.

Nicholas Carlini and David Wagner. Towards evaluating the robustness of neural networks. In 2017 ieee symposium on security and privacy (sp), pp. 39–57. Ieee, 2017.

Erh-Chung Chen and Che-Rung Lee. LTD: low temperature distillation for robust adversarial training. CoRR, abs/2111.02331, 2021. URL https://arxiv.org/abs/2111.02331.

Tianlong Chen, Zhenyu Zhang, Sijia Liu, Shiyu Chang, and Zhangyang Wang. Robust overfitting may be mitigated by properly learned smoothening. In International Conference on Learning Representations, 2021.

Minhao Cheng, Qi Lei, Pin-Yu Chen, Inderjit Dhillon, and Cho-Jui Hsieh. Customized adversarial training for improved robustness. In IJCAI, 2022.

Seungju Cho and Changick Kim. Direction-preserving fast adversarial training. In under review, 2026.

Francesco Croce and Matthias Hein. Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks. In International conference on machine learning, pp. 2206–2216. PMLR, 2020.

Jiequan Cui, Shu Liu, Liwei Wang, and Jiaya Jia. Learnable boundary guided adversarial training. In ICCV, 2021.

Gavin Weiguang Ding, Yash Sharma, Kry Yik Chau Lui, and Ruitong Huang. MMA training: Direct input space margin maximization through adversarial training. In ICLR, 2020.

Yinpeng Dong, Ke Xu, Xiao Yang, Tianyu Pang, Zhijie Deng, Hang Su, and Jun Zhu. Exploring memorization in adversarial training. In International Conference on Learning Representations, 2022.

Micah Goldblum, Liam Fowl, Soheil Feizi, and Tom Goldstein. Adversarially robust distillation. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 34, pp. 3996–4003, 2020.

Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. arXiv preprint arXiv:1412.6572, 2014.

Shruthi Gowda, Bahram Zonooz, and Elahe Arani. Conserve-update-revise to cure generalization and robustness trade-off in adversarial training. In ICLR, 2024.

Sorin Grigorescu, Bogdan Trasnea, Tiberiu Cocias, and Gigel Macesanu. A survey of deep learning techniques for autonomous driving. Journal of Field Robotics, 37(3):362–386, 2020.

Bo Huang, Mingyang Chen, Yi Wang, Junda Lu, Minhao Cheng, and Wei Wang. Boosting accuracy and robustness of student models via adaptive adversarial distillation. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 24668–24677, 2023.

Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Logan Engstrom, Brandon Tran, and Aleksander Madry. Adversarial examples are not bugs, they are features. In NeurIPS, 2019.

Pavel Izmailov, Dmitrii Podoprikhin, Timur Garipov, Dmitry Vetrov, and Andrew Gordon Wilson. Averaging weights leads to wider optima and better generalization. In Conference on Uncertainty in Artificial Intelligence, 2018.

Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.

Ya Le and Xuan Yang. Tiny imagenet visual recognition challenge. CS 231N, 7(7):3, 2015.

Hongsin Lee, Seungju Cho, and Changick Kim. Indirect gradient matching for adversarial robust distillation. In ICLR, 2025.

Yanyun Liu et al. Failure cases are better learned but boundary says sorry: Facilitating smooth perception change for accuracy-robustness trade-off in adversarial training. In ICCV, 2025.

Xingjun Ma, Yuhao Niu, Lin Gu, Yisen Wang, Yitian Zhao, James Bailey, and Feng Lu. Understanding adversarial attacks on deep learning based medical image analysis systems. Pattern Recognition, 110:107332, 2021.

Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. arXiv preprint arXiv:1706.06083, 2017.

Tianyu Pang, Xiao Yang, Yinpeng Dong, Hang Su, and Jun Zhu. Bag of tricks for adversarial training. arXiv preprint arXiv:2010.00467, 2020.

Leslie Rice, Eric Wong, and J. Zico Kolter. Overfitting in adversarially robust deep learning. In ICML, 2020.

Satoshi Suzuki, Shin'ya Yamaguchi, Shoichiro Takeda, Sekitoshi Kanai, Naoki Makishima, Atsushi Ando, and Ryo Masumura. ARREST: Adversarial finetuning with latent representation constraint to mitigate the accuracy-robustness tradeoff. In ICCV, 2023.

Jihoon Tack, Sihyun Yu, Jongheon Jeong, Minseon Kim, Sung Ju Hwang, and Jinwoo Shin. Consistency regularization for adversarial robustness. In Proceedings of the AAAI Conference on Artificial Intelligence, volume 36, pp. 8414–8422, 2022.

Dimitris Tsipras, Shibani Santurkar, Logan Engstrom, Alexander Turner, and Aleksander Madry. Robustness may be at odds with accuracy. arXiv preprint arXiv:1805.12152, 2018.

Yisen Wang et al. Robust weight perturbation for adversarial training. In NeurIPS, 2023a.

Zekai Wang, Tianyu Pang, Chao Du, Min Lin, Weiwei Liu, and Shuicheng Yan. Better diffusion models further improve adversarial training. In International Conference on Machine Learning (ICML), 2023b.

Dongxian Wu, Shu-Tao Xia, and Yisen Wang. Adversarial weight perturbation helps robust generalization. Advances in Neural Information Processing Systems, 33:2958–2969, 2020.

Yu-Yu Wu, Hung-Jui Wang, and Shang-Tse Chen. Annealing self-distillation rectification improves adversarial training. In ICLR, 2024.

Hongyang Zhang, Yaodong Yu, Jiantao Jiao, Eric Xing, Laurent El Ghaoui, and Michael Jordan. Theoretically principled trade-off between robustness and accuracy. In International conference on machine learning, pp. 7472–7482. PMLR, 2019.

Shiji Zhao, Xizhe Wang, and Xingxing Wei. Mitigating accuracy-robustness trade-off via balanced multi-teacher adversarial distillation. IEEE TPAMI, 2024.

Jianing Zhu, Jiangchao Yao, Bo Han, Jingfeng Zhang, Tongliang Liu, Gang Niu, Jingren Zhou, Jianliang Xu, and Hongxia Yang. Reliable adversarial distillation with unreliable teachers. arXiv preprint arXiv:2106.04928, 2021.

Bojia Zi, Shihao Zhao, Xingjun Ma, and Yu-Gang Jiang. Revisiting adversarial robustness distillation: Robust soft labels make student better. In Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 16443–16452, 2021.

Table 2: Main results. ResNet-18, ℓ_∞, $\epsilon = 8/255$. Every number is measured in this framework—one data pipeline, one attack implementation, one evaluation, the final-epoch weight-averaged model—so the rows are comparable to each other rather than across papers, and no source column is needed. Teacher is what the method assumes available: – none, self its own weight average, nat. a naturally trained network. Rows we have not run are left empty with the reason. Avg $= \frac{1}{2}$(Clean + AA), the criterion B-MTARD calls W-Robust; NRR is the harmonic mean. [j] LBGAT trains its natural branch jointly, so its teacher is not free. [n] The CIFAR-100 cell is LBGAT6 and the CIFAR-10 cell is LBGAT0, which is what the authors publish for each: their release lists CIFAR-10 checkpoints under LBGAT0 only, and $\beta = 6$ is their CIFAR-100 setting. We report the variant each dataset has a published configuration for rather than forcing one β across both. [o] HAT is run on its own one-cycle schedule rather than ours, which collapses it to chance on both datasets; the same allowance is made for LBGAT. On CIFAR-10 our port reproduces HAT's published no-extra-data figure to within 0.31 clean and 0.27 AutoAttack (84.90/49.08 reported, ResNet-18). HAT publishes no CIFAR-100 result without additional data — its 61.50/28.88 is trained with DDPM-generated data — so the CIFAR-100 column here has no published counterpart to read it against. [m] ADR is being run on separate hardware and its row will be filled from there. Two things have to hold when it is. ADR reports two configurations, AT + ADR (56.10/26.87 on CIFAR-100) and AT + WA + AWP + ADR (57.36/28.50), and the row must name which one it is — the second is the one component-matched to ours, since our recipe carries both weight averaging and AWP. And ADR is self-distillation: its teacher is an exponential moving average of the student, no external network is read, and the student trains from scratch, so the row inherits nothing of our natural teacher and should not be grouped with the methods that do. [l] HAT uses a natural network only to label helper points beyond the ball, never as a target, so it is grouped with the trade-off methods. [k] Not a published method: our own recipe with the feature anchor replaced by logit KL. It is the closest thing to KD + SWA (Chen et al., 2021) that a natural teacher admits — distil, then average weights, with a logit target where ours is a feature one — and it isolates the target from the schedule. The temperature is chosen in the baseline's favour rather than faithfully: plain distillation would be $\tau = 1$, which is by some margin the worst of the three in Table 9 at 20.84 AutoAttack, and this row instead uses the τ that maximizes it, 4 at 24.48 there. The comparison is therefore against the best logit target we could find and not against the one the name implies. At the full recipe the temperature matters far more than it does in the base regime, where the two sit 0.48 apart: $\tau = 16$ reaches only 42.45/20.10 against $\tau = 4$'s 56.97/26.96. [‡] IGDM is reported at $\alpha = 1$; its published $\alpha = 20$ diverges to chance accuracy on both datasets when the teacher is natural, for the reason given in Section 5.3. Published figures for these methods are in Table 14, Tiny-ImageNet in Table 3, WideResNet in Table 4, and per-attack metrics in Table 13.

		CIFAR-10				CIFAR-100				
Method	T.	Clean	AA	Avg	NRR	Clean	AA	Avg	NRR	
PGD-AT	–	–	–	–	–	–	–	–	–	other machine
TRADES	–	–	–	–	–	–	–	–	–	other machine
MART	–	–	–	–	–	–	–	–	–	other machine
Consistency-AT	–	–	–	–	–	–	–	–	–	other machine
PGD-AT + WA	–	–	–	–	–	–	–	–	–	other machine
PGD-AT + WA + AWP	–	–	–	–	–	–	–	–	–	other machine
PGD-AT, init. at the teacher	nat.	82.23	50.72	66.47	62.74	57.73	26.46	42.09	36.29	
ARD	nat.	84.58	43.75	64.16	57.67	57.61	20.24	38.92	29.96	
RSLAD	nat.	85.15	42.91	64.03	57.06	59.68	21.30	40.49	31.40	
AdaAD	nat.	85.39	46.22	65.81	59.98	59.79	23.19	41.49	33.42	
AdaAD + IGDM[‡]	nat.	85.01	45.50	65.25	59.27	48.25	19.48	33.87	27.75	
Logit anchor + our stack[k]	nat.	–	–	–	–	56.97	26.96	41.97	36.60	C10 not run
LBGAT[n]	nat.[j]	–	–	–	–	56.46	26.02	41.24	35.62	C10 running
ARREST	nat.	–	–	–	–	–	–	–	–	not run
HAT[o]	nat.[l]	84.59	48.81	66.70	61.90	60.67	24.45	42.56	34.85	
ADR[m]	self	–	–	–	–	–	–	–	–	other machine
CURE	self	–	–	–	–	–	–	–	–	no repro.

Table 3: Tiny-ImageNet-200, ResNet-18. Same eleven rows and the same protocol as Table 2; – marks a cell we have not run — the baseline rows for this dataset are being produced on separate hardware and will be filled from there. Our row uses the 200-epoch teacher; the 80-epoch teacher gives 57.08/18.96, which is the teacher-length dial of Table 12 on a third dataset. Our row is above every published ResNet-18 Tiny-ImageNet result on both axes: the highest published clean accuracy is HAT's 52.60 and the highest AutoAttack is 20.23 (AT + WA + ADR), against our 55.16/20.54. Table 15 collects them.

Method	Teacher	Clean	AA	Avg	NRR
PGD-AT	–	–	–	–	–
TRADES	–	–	–	–	–
MART	–	–	–	–	–
Consistency-AT	–	–	–	–	–
PGD-AT + WA	–	–	–	–	–
PGD-AT + WA + AWP	–	–	–	–	–
PGD-AT, init. at the teacher	nat.	–	–	–	–
ARD	nat.	–	–	–	–
RSLAD	nat.	–	–	–	–
AdaAD	nat.	–	–	–	–
AdaAD + IGDM	nat.	–	–	–	–
Logit anchor + our stack[k]	nat.	–	–	–	–
LBGAT	nat.[j]	–	–	–	–
ARREST	nat.	–	–	–	–
HAT	nat.[l]	–	–	–	–
ADR	self	–	–	–	–
CURE	self	–	–	–	–
RPAT++	–	–	–	–	–
CFA (ours)	nat.	55.16	20.54	37.85	29.93

Table 4: WideResNet-34-10, ℓ_∞, $\epsilon = 8/255$. Same eleven rows and the same protocol as Table 2; – marks a cell we have not run — as for Table 3, this architecture is being run on separate hardware, our CIFAR-100 row included. Comparison lines are in Table 14: on CIFAR-10 the strongest published pair on either criterion is AT + WA + AWP at 87.42/55.01 (Avg 71.22, NRR 67.53), and on CIFAR-100 it is ADR's 62.21/31.60 (NRR 41.91) against ARREST's 73.05/24.32 (Avg 48.69).

		CIFAR-10			CIFAR-100		
Method	T.	Clean	AA	NRR	Clean	AA	NRR
PGD-AT	–	–	–	–	–	–	–
TRADES	–	–	–	–	–	–	–
MART	–	–	–	–	–	–	–
Consistency-AT	–	–	–	–	–	–	–
PGD-AT + WA	–	–	–	–	–	–	–
PGD-AT + WA + AWP	–	–	–	–	–	–	–
PGD-AT, init. at the teacher	nat.	–	–	–	–	–	–
ARD	nat.	–	–	–	–	–	–
RSLAD	nat.	–	–	–	–	–	–
AdaAD	nat.	–	–	–	–	–	–
AdaAD + IGDM	nat.	–	–	–	–	–	–
Logit anchor + our stack[k]	nat.	–	–	–	–	–	–
LBGAT	nat.[j]	–	–	–	–	–	–
ARREST	nat.	–	–	–	–	–	–
HAT	nat.[l]	–	–	–	–	–	–
ADR	self	–	–	–	–	–	–
CURE	self	–	–	–	–	–	–
RPAT++	–	–	–	–	–	–	–
CFA (ours)	nat.	88.67	55.29	68.11	in progress		

Table 5: The four published distillation objectives given the teacher they would have in our setting—our naturally trained network in place of the adversarially trained one they assume. Every number is measured here: ResNet-18, 100 epochs, each method at its own published recipe, same data pipeline and evaluation as every other row we report. The reference rows use that same teacher for initialization only, or as an anchor. Blank cells are runs still in progress.

	CIFAR-100			CIFAR-10		
	Clean	AA	NRR	Clean	AA	NRR
ARD	57.61	20.24	29.96	84.58	43.75	57.67
RSLAD	59.68	21.30	31.40	85.15	42.91	57.06
AdaAD	59.79	23.19	33.42	85.39	46.22	59.98
AdaAD + IGDM‡	48.25	19.48	27.75	85.01	45.50	59.27
PGD-AT, initialized at the teacher	57.73	26.46	36.29	82.23	50.72	62.74
CFA (ours), no WA/AWP	61.21	25.24	35.74	–	–	–

Table 6: Component ablation. CIFAR-100, ResNet-18, raw-ℓ_2 anchor with $\epsilon_{\text{train}} = 8.8/255$ held fixed and no learning-rate freeze. Each row adds one component to the row above, and every rung inherits the teacher's classifier without training it, as the shipped recipe does — so the 100-epoch ladder ends exactly on the row Table 2 reports. Columns match Table 2; per-attack values are in Table 13. The 50-epoch block is still the earlier build, in which every rung trains the head, and is being rebuilt; until it is, deltas should be read within a block and not across the two.

	Clean	AA	Avg	NRR
50 epochs				
Anchor ($p = 0$)	61.33	26.19	43.76	36.71
+ sensitivity-matched ϵ	62.94	26.40	44.67	37.20
+ weight averaging	61.11	28.06	44.59	38.46
+ AWP†	59.90	28.05	43.98	38.21
100 epochs				
Anchor ($p = 0$)	61.29	25.36	43.33	35.88
+ sensitivity-matched ϵ	63.09	25.74	44.42	36.56
+ weight averaging	62.30	27.69	45.00	38.34
+ AWP†	62.17	28.86	45.52	39.42

Table 7: Two controls on the anchor itself. CIFAR-100, ResNet-18, base regime—teacher warm start, no weight averaging, no AWP, $p = 0$, $\epsilon_{\text{train}} = 8.8/255$—so that no stack component can absorb the difference. Each row differs from its reference in exactly one line of configuration.

	Clean	PGD-20	CW	AA	NRR
(a) where the teacher is read (50 epochs)					
Anchor, Φ_t read at x	61.33	32.94	27.80	26.19	36.71
read at x_{adv} instead	76.11	0.00	0.00	0.00	0.00
(b) the anchor against label CE, same regime (100 epochs)					
Anchor	61.21	31.26	26.82	25.24	35.74
label cross-entropy instead	54.60	20.60	21.21	19.45	28.68

Table 8: Local linearity, which IGDM's construction requires of the teacher: remainder proportion of the first-order Taylor expansion over an $\epsilon = 8/255$ ball, computed as in IGDM's Sec. 3.1.

Network	Remainder proportion
IGDM's robust teachers (LTD / BDM-AT / IKL-AT, published)	0.012 / 0.012 / 0.016
PGD-AT ResNet-18 (ours)	0.0111
Natural teacher (ours)	0.4763
Anchored student, $p = 0$	0.0029
Anchored student, shipped recipe	0.0202

Table 9: What to do with the classifier. CIFAR-100 base regime (50 epochs, raw features, no weight averaging, no AWP, $\epsilon = 8/255$), so that no stack component can absorb the difference.

	Clean	PGD-20	CW	AA	NRR
Logit target only, $\tau = 1$	58.26	22.62	22.79	20.84	30.70
Logit target only, $\tau = 4$	59.39	30.30	26.84	24.48	34.67
Logit target only, $\tau = 16$	57.78	31.57	26.11	24.00	33.91
Feature anchor + head KD, $\tau = 1$	62.34	26.34	26.11	23.93	34.58
Feature anchor + head KD, $\tau = 16$	62.61	32.47	27.32	25.61	36.35
Feature anchor, head untouched	62.72	28.69	27.80	25.88	36.64
head refit, adversarial CE	62.77	29.93	27.66	25.65	36.42
head refit, adv. CE + smoothing 0.1	62.57	30.43	27.55	25.66	36.39
head refit, clean CE	62.70	28.90	27.23	25.15	35.90

Table 10: ϵ_{train} swept with everything else fixed. Evaluation is at $\epsilon = 8/255$ in every cell.

	Clean	PGD-20	CW	AA	NRR
CIFAR-100					
6/255	68.07	28.40	28.16	25.82	37.44
7/255	65.61	30.34	29.64	27.44	38.70
8/255	63.89	31.41	29.83	27.78	38.72
$8.8/255^{\dagger}$	62.17	32.37	30.93	28.86	39.42
$10/255^{\dagger}$	59.99	32.82	30.53	28.77	38.89
CIFAR-10					
6/255	89.64	49.29	50.20	47.88	62.42
7/255	88.30	51.43	52.64	50.27	64.07
$8/255^{\dagger}$	87.22	52.43	53.55	51.15	64.48
$8.8/255^{\dagger}$	84.96	52.98	53.94	51.74	64.31
$10/255^{\dagger}$	83.29	53.25	53.88	51.79	63.87

Table 11: What the per-sample radius is allocated from. CIFAR-100, ResNet-18, 100 epochs, $\epsilon_{\text{train}} = 8.8/255$, the shipped recipe throughout; every cell differs from the one above it in a single configuration line and nothing else. All of them spend the same total budget $N\epsilon_{\text{train}}$, so the rows differ in allocation and not in attack strength. sd(w) is the spread of the per-sample multiplier the attack actually receives — after the box and the mean restoration, so it is comparable across rows in a way the raw signals are not — reported as the median over training; the permuted row reuses the sensitivity rule's own weights, hence its value. Only the sensitivity signal beats allocating nothing at all — the three signals the existing per-sample family allocates from each land below the uniform row, and the most dispersed of them lands lowest.

Allocated from	sd(w)	Clean	PGD-20	CW	AA	NRR
Nothing (uniform, $p = 0$)	0	60.42	33.11	30.07	28.42	38.66
Loss sensitivity (ours, $p = 1$)	0.32	62.17	32.37	30.93	28.86	39.42
Difficulty magnitude (IAAT, CAT)	0.39	61.27	32.45	30.14	28.12	38.55
Difficulty, same weights permuted	0.32	60.90	32.32	29.94	28.06	38.42
Logit margin (MMA)	0.42	61.07	32.46	29.75	27.79	38.20

Table 12: Only the teacher checkpoint changes; the student configuration is untouched. CIFAR-100, ResNet-18. S_w/S_b is the ratio of within-class to between-class scatter on the unit sphere.

Teacher	Teacher clean	Teacher S_w/S_b	Student clean	Student AA	NRR
50 epochs	75.81	1.100	64.28	24.38	35.35
100 epochs	76.62	0.982	63.65	25.20	36.11
150 epochs	77.52	0.887	62.93	25.78	36.51
200 epochs	77.65	0.808	62.72	25.88	36.64
300 epochs	78.32	0.712	62.24	25.80	36.48

Table 13: Per-attack accuracy (%) for our cells. ResNet-18, $\epsilon = 8/255$, final-epoch weight-averaged model, full test set.

Dataset	Cell	Clean	FGSM	PGD-20	CW	AA	NRR
CIFAR-100	CFA, head frozen (shipped)	62.17	36.91	32.37	30.93	28.86	39.42
	CFA, head-KD term retained	62.35	38.80	36.26	30.65	28.68	39.29
	CFA, $\epsilon_{\text{tr}} = 8/255$	63.89	36.67	31.41	29.83	27.78	38.72
	CFA, $\epsilon_{\text{tr}} = 10/255$	59.99	36.50	32.82	30.53	28.77	38.89
	CFA, no WA / AWP	61.21	35.54	31.26	26.82	25.24	35.74
	RPAT++ (our reproduction)	55.93	34.46	32.44	29.37	27.36	36.74
CIFAR-10	CFA, $\epsilon_{\text{tr}} = 8.8/255$ (shipped)	84.96	59.58	52.98	53.94	51.74	64.31
	CFA, $\epsilon_{\text{tr}} = 8/255$	87.22	59.92	52.43	53.55	51.15	64.48
	CFA, $\epsilon_{\text{tr}} = 10/255$	83.29	58.98	53.25	53.88	51.79	63.87
	RPAT++ (our reproduction)	82.41	59.73	55.72	52.88	50.75	62.82

Table 14: Published results, ℓ_∞, $\epsilon = 8/255$, quoted as reported. The architecture changes between blocks and is named in each block heading: the first two are ResNet-18 and are the ones comparable with Table 2, the next two are PreActResNet-18 and the last is WideResNet-34-10. RPAT's leaderboard is reported on the latter two, so its rows are not directly comparable with our ResNet-18 numbers—RPAT++ appears at 82.63/51.00 on PreActResNet-18 CIFAR-10 and at 86.76/54.97 on WideResNet-34-10, and the WideResNet figure is the comparison line for Table 4, not for Table 2.

Method	Teacher	Clean	AA	Avg	NRR	Source
CIFAR-100, ResNet-18						
PGD-AT	–	56.56	25.02	40.79	34.69	RPAT
TRADES	–	55.39	24.51	39.95	33.98	RPAT
MART	–	49.83	25.00	37.41	33.30	RPAT
Consistency-AT	–	58.53	25.39	41.96	35.42	RPAT
Consistency-AT + RPAT	–	60.33	26.31	43.32	36.64	RPAT
F^2AT	–	54.19	23.24	38.71	32.53	F^2AT
Generalist++	co-tr.	62.97	23.96	43.47	34.71	Generalist++
ADR (AT + ADR)	self	56.10	26.87	41.48	36.34	ADR
ADR (AT + WA + AWP + ADR)	self	57.36	28.50	42.93	38.08	ADR
CIFAR-10, ResNet-18						
PGD-AT	–	82.78	44.63	63.71	57.99	CURE
TRADES	–	82.41	48.37	65.39	60.96	CURE
MART	–	80.70	47.49	64.10	59.79	CURE
ST-AT	–	83.10	50.50	66.80	62.82	CURE
IAD	robust	80.63	50.17	65.40	61.85	CURE
Consistency-AT + RPAT	–	84.12	48.98	66.55	61.91	RPAT
Generalist++	co-tr.	89.09	46.07	67.58	60.73	Generalist++
ARREST	natural	86.63	46.14	66.39	60.21	ARREST
CURE	self	86.76	49.69	68.23	63.19	CURE
ADR (AT + ADR)	self	82.41	50.38	66.40	62.53	ADR
CIFAR-10, PreActResNet-18						
WA (Izmailov et al., 2018)	–	83.50	49.89	66.70	62.46	RPAT
AWP (Wu et al., 2020)	–	81.11	50.09	65.60	61.93	RPAT
KD + SWA (Chen et al., 2021)	robust	84.06	49.82	66.94	62.56	RPAT
TE (Dong et al., 2022)	–	82.04	50.12	66.08	62.23	RPAT
ReBAT (Wang et al., 2023a)	–	82.09	50.72	66.41	62.70	RPAT
RPAT++ (Liu et al., 2025)	–	82.63	51.00	66.82	63.07	RPAT
CIFAR-100, PreActResNet-18						
WA (Izmailov et al., 2018)	–	57.26	25.83	41.55	35.60	RPAT
AWP (Wu et al., 2020)	–	54.10	25.16	39.63	34.35	RPAT
KD + SWA (Chen et al., 2021)	robust	57.17	25.66	41.42	35.42	RPAT
TE (Dong et al., 2022)	–	56.41	25.84	41.13	35.44	RPAT
ReBAT (Wang et al., 2023a)	–	56.13	27.60	41.87	37.00	RPAT
RPAT++ (Liu et al., 2025)	–	56.84	27.68	42.26	37.23	RPAT
CIFAR-10, WideResNet-34-10						
WA (Izmailov et al., 2018)	–	87.66	52.65	70.16	65.79	RPAT
MMA (Ding et al., 2020)	–	87.80	43.10	65.45	57.82	RPAT
AWP (Wu et al., 2020)	–	85.63	53.32	69.48	65.72	RPAT
KD + SWA (Chen et al., 2021)	robust	87.45	53.59	70.52	66.46	RPAT
TE (Dong et al., 2022)	–	85.97	52.88	69.43	65.48	RPAT
ReBAT (Wang et al., 2023a)	–	85.25	54.78	70.02	66.70	RPAT
ADR (Wu et al., 2024)	self	84.67	53.25	68.96	65.38	RPAT
CURE (Gowda et al., 2024)	self	87.05	52.10	69.58	65.19	RPAT
RPAT++ (Liu et al., 2025)	–	86.76	54.97	70.87	67.30	RPAT

Table 15: Published Tiny-ImageNet-200 results, ResNet-18, ℓ_∞, $\epsilon = 8/255$, quoted as reported. Our row is strictly above all of them on both axes, and above all thirteen rows that are the same network as ours. ADR trains this dataset for 80 epochs against our 100, which favours us and is stated here rather than left implicit. Rows from F^{2}AT and RPAT are separated because neither is comparable. RPAT's "ResNet-18" is ResNet(Bottleneck, [2,2,2,2]) in its released code, 14.35M parameters against the 11.27M of the standard BasicBlock network used by us and by ADR—they also ship a pre_resnet18 at 11.27M and use PreActResNet-18 explicitly elsewhere, so the distinction is theirs. F^{2}AT's plain SAT baseline reaches 31.52/9.37 where ADR's plain AT reaches 45.87/18.06, a gap no difference of method explains. HAT's row is separated for a different reason: ADR lists it under ResNet-18, but the numbers are quoted from HAT's own Table 9, captioned "using PreAct ResNet-18", and HAT's code refuses anything but a PreAct network on this dataset. Same 11.27M parameters as ours, different network, so its two companions from that table are shown with it. *TE's architecture we could not verify — it is ADR's attribution to a paper we do not have. LBGAT reports this dataset without AutoAttack and B-MTARD reports it on MobileNet-v2, so neither appears; CURE, ARREST and Generalist++ report it not at all.

Method	Clean	AA	Avg	NRR	Source
CFA (ours)	55.16	20.54	37.85	29.93	—
AT + WA + ADR	48.55	20.23	34.39	28.56	ADR
AT + WA + AWP + ADR	48.27	20.12	34.20	28.40	ADR
TRADES + WA + AWP + ADR	51.38	19.48	35.43	28.25	ADR
TRADES + WA + ADR	51.99	19.17	35.58	28.01	ADR
AT + WA + AWP	48.61	19.58	34.09	27.92	ADR
AT + WA	49.10	19.30	34.20	27.71	ADR
TE*	47.46	18.29	32.88	26.40	ADR
AT	45.87	18.06	31.96	25.92	ADR
TRADES	48.49	17.35	32.92	25.56	ADR
PreActResNet-18 — same parameter count, different network					
HAT	52.60	18.14	35.37	26.98	HAT
TRADES	48.25	17.17	32.71	25.35	HAT
AT	47.76	17.92	32.84	26.02	HAT
Bottleneck ResNet-18 (14.35M) and an unstated setup — see caption					
Consistency-AT + RPAT	49.74	18.84	34.29	27.33	RPAT
PGD-AT + RPAT	47.68	17.77	32.73	25.89	RPAT
Consistency-AT	46.54	17.60	32.07	25.54	RPAT
MART	39.70	17.18	28.44	23.98	RPAT
F^{2}AT	40.54	13.13	26.84	19.84	F^{2}AT
SEAT	35.51	11.37	23.44	17.22	F^{2}AT
SAT	31.52	9.37	20.45	14.45	F^{2}AT